2Time0.0001 s 4Time0.0 s 8Time0.0 s 12Time0.0 s 16Time0.0 s 20Time0.0 s

Text and String Encodings: SMT Theories for Python String Verification

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

String manipulation is pervasive in Python programs: identifiers follow naming conventions, format strings carry structural constraints, documentation embeds preconditions, and streaming pipelines process text in chunks. Yet string-heavy programs are notoriously difficult to verify using arithmetic-only solvers. We present TextEnc, the string encoding layer of the JuGeo verification framework, which translates Python `str` operations into the QF_SLIA SMT theory understood by Z3. The layer comprises three cooperating components: (i) a *string model* (StrEnc/SymStr) that captures Python's `str` semantics including Unicode normalisation, (ii) a *streaming encoding* (StreamEnc) that handles string processing in chunks, and (iii) a *family classifier* that routes each constraint to its least complex decidable fragment. We state and prove a *String Encoding Completeness Theorem*: every Python string operation has a corresponding QF_SLIA encoding that preserves equality semantics. Experiments on 300 Python functions demonstrate that TextEnc reduces average query time compared to naïve bit-vector encodings while maintaining full soundness.

Contents

1 Introduction

Python's `str` type is one of the most heavily used data types in the language. Programs name variables with identifiers, route HTTP requests via URL strings, persist data through serialisation formats, and generate user output via f-strings and `format` templates. The semantics of `str` operations—concatenation, slicing, regular-expression matching, Unicode normalisation—are precisely specified in the Python language reference [1], yet most verification tools treat strings as uninterpreted blobs or fall back to costly bit-vector models.

The string verification gap. Arithmetic SMT theories (`QF_LIA`, `QF_LRA`) express numeric invariants with great precision. Bit-vector theories (`QF_BV`) model machine-word arithmetic. The SMT community has more recently standardised the *theory of strings* (QF_S/QF_SLIA) [2], which Z3 [3] and CVC5 [4] implement. However, the gap between Python's rich string semantics and the raw SMT string theory primitives is non-trivial: Python strings are Unicode sequences (not byte arrays), `str.find` returns -1 on failure (not an exception), and f-string formatting composes multiple operations that must be encoded atomically. Bridging this gap requires a carefully engineered encoding layer.

Contributions. This paper makes the following contributions.

1. **String Model (§2)**: We define the StrEnc and SymStr data structures that capture Python `str` semantics in QF_SLIA (§2).
2. **Core Operations (§3)**: We give precise QF_SLIA encodings for concatenation, slicing, `find`/`index`, `replace`, and format strings.
3. **Streaming Encoding (§4)**: We introduce StreamEnc, which processes strings in chunks while preserving whole-string invariants.
4. **Unicode Handling (§5)**: We explain how the `NormalizedTextEnvironment` enforces NFC invariants in the encoding.
5. **Regex Integration (§6)**: We show how Python regex patterns are compiled to QF_SLIA regular-expression terms.
6. **Completeness Theorem (§7)**: We prove that every Python string operation has a corresponding QF_SLIA encoding that preserves equality semantics, and provide the Lean 4 formalisation in `Paper33_TextEncodings.lean`.
7. **Evaluation (§8)**: We measure overhead and solver performance on the JuGeo benchmark suite.

Relation to earlier papers. Papers 1–5 established the judgment 8-tuple (c, φ, $\mathcal{A}$, $\mathcal{E}$, $\mathcal{O}$, $\mathcal{B}$, $\mathcal{T}$, Π) and trust algebra. Papers 6–10 gave Python-specific encodings for arithmetic, effects, and proof-carrying contracts. Papers 11–32 applied the framework to advanced algebraic and geometric settings. The present paper focuses on the *string* component of $\mathcal{E}$ and $\mathcal{O}$, which previous papers treated only lightly.

2 String Model

2.1 Python String Semantics

Python's `str` is an immutable sequence of Unicode code points. Unlike C strings, it carries no null terminator; unlike Java `String`, it has no UTF-16 surrogate-pair artefacts. The key semantic properties we must capture are:

1. **Unicode**: every character is a code point $c \in [0, 0x10\text{FFFF}]$.
2. **Immutability**: operations return new strings; no in-place mutation.
3. **Length**: $\mathsf{str.len}(s)$ returns the number of code points, not bytes.
4. **Slicing**: `s[i:j]` equals $\mathsf{str.substr}(s, i, j - i)$ with clamping semantics for out-of-bounds indices.
5. **Find**: `s.find(t)` returns the first index i such that $\mathsf{str.substr}(s, i, \mathsf{str.len}(t)) = t$, or -1 if absent.
6. **Replace**: `s.replace(old, new)` replaces the first (or all, with `count`) occurrences.
7. **Format**: f-strings and `str.format` compose substrings and conversions.

The QF_SLIA theory provides sorts `String` and `Int` with mixed string-integer operators. The key SMT-LIB2 primitives we use are listed in Table 1.

Table 1: SMT-LIB2 string primitives used by TextEnc.

SMT Primitive	Python Equivalent	Fragment
$\mathsf{str.len}(s)$	`len(s)`	QF_SLIA
$\mathsf{str.prefixof}(p, s)$	`s.startswith(p)`	QF_S
$\mathsf{str.suffixof}(p, s)$	`s.endswith(p)`	QF_S
$\mathsf{str.contains}(s, t)$	`t in s`	QF_S
$\mathsf{str.in\_re}(s, r)$	`re.fullmatch(r, s)`	QF_S
$s +\!\!+ t$	`s + t`	QF_S
$\mathsf{str.substr}(s, i, n)$	`s[i:i+n]`	QF_SLIA
$\mathsf{str.indexof}(s, t)$	`s.find(t)`	QF_SLIA
$\mathsf{str.replace}(s, \mathit{old}, \mathit{new})$	`s.replace(old, new)`	QF_S

2.2 The `StrEnc` Data Structure

Definition 2.1 (String Encoding)**.** A *string encoding* is a frozen record

$$\mathsf{StrEnc} = \big(v,\ z,\ \ell^{?},\ \vec{p},\ \vec{s},\ \vec{r},\ \pi\big)$$

where:

- $v \in \Sigma^*$ is the concrete string value (may be empty for a purely symbolic encoding),
- z is the SMT-LIB2 variable name for the string sort,

- $\ell^? \in \mathbb{Z}_{\geq 0} \cup \{\bot\}$ is an optional explicit length constraint,
- $\vec{p} = (p_1, \ldots, p_k)$ are required prefixes,
- $\vec{s} = (s_1, \ldots, s_m)$ are required suffixes,
- $\vec{r} = (r_1, \ldots, r_n)$ are regular-expression membership constraints, and
- π is a provenance tag for debugging.

The *assertion* of StrEnc is the conjunction

$$\phi(\mathsf{StrEnc}) \;:=\; \big(\ell^? \neq \bot \Rightarrow \mathsf{str.len}(z) = \ell^?\big) \wedge \bigwedge_i \mathsf{str.prefixof}(p_i,\, z) \wedge \bigwedge_j \mathsf{str.suffixof}(s_j,\, z) \wedge \bigwedge_k \mathsf{str.in\_re}(z,\, r_k).$$

StrEnc is implemented as a frozen Python `dataclass` so that instances can be shared across solver sessions without accidental mutation.

2.3 Symbolic Text Variables

When the concrete value is not known at encoding time—for example, a function parameter whose value is determined by the caller—we use *symbolic text* variables.

Definition 2.2 (Symbolic Text)**.** A *symbolic text variable* is a mutable record

$$\mathsf{SymStr} = \big(x,\; \sigma,\; \lambda^?,\; [\ell_{\min}, \ell_{\max}],\; \vec{c},\; \vec{q},\; \nu^?\big)$$

where:

- x is the SMT-LIB2 variable name,
- σ is the SMT-LIB2 sort (always `String` here),
- $\lambda^?$ is an optional naming law (Definition 2.3),
- $[\ell_{\min}, \ell_{\max}]$ are length bounds with $\ell_{\min} \leq \ell_{\max}$,
- $\vec{c}$ are character-class regex constraints,
- $\vec{q}$ are high-level semantic assertions (strings),
- $\nu^?$ is an optional Unicode normalisation requirement.

The method `SymStr.to_z3_formula()` compiles the record to an SMT-LIB2 fragment:

```
(declare-const x String)
(assert (>= (str.len x) l_min))
(assert (<= (str.len x) l_max))
(assert (str.in_re x (re.* (re.range "a" "z"))))  ; character class
```

2.4 Naming Laws

Python codebases enforce naming conventions (`snake_case`, `PascalCase`, UUID format, semantic-versioning tags, etc.). These form constraints on identifiers that appear in the proof obligations.

Definition 2.3 (Naming Law)**.** A *naming law* is a frozen record $\mathsf{NamLaw} = (\mathit{name}, \rho, \vec{f}, \ell_{\min}, \ell_{\max}, \vec{a})$ where ρ is a primary regex, $\vec{f}$ are forbidden sub-patterns, $\ell_{\min}/\ell_{\max}$ are length bounds, and $\vec{a}$ are SMT-LIB2 axiom strings.

The four standard laws are:

- `snake_case`: $\rho =$ `[a-z][a-z0-9_]*`, length $[1, 80]$.
- `camelCase`: $\rho =$ `[a-z][a-zA-Z0-9]*`, length $[1, 80]$.
- `uuid`: $\rho =$ `[0-9a-f]{8}-...-[0-9a-f]{12}`, length $[36, 36]$.
- `semver`: $\rho =$ `[0-9]+[0-9]+[0-9]+`, length $[5, 30]$.

3 Core String Operations

3.1 Concatenation

Python `s + t` and `f"st"` both reduce to string concatenation. In QF_SLIA:

$$\mathsf{enc}(s + t) \;:=\; z_{res} = z_s +\!\!+ z_t,$$

where z_s, z_t, z_{res} are fresh `String` variables and $z_s \mapsto \mathsf{StrEnc}_s$, $z_t \mapsto \mathsf{StrEnc}_t$ are already declared. Length propagates as $\mathsf{str.len}(z_{res}) = \mathsf{str.len}(z_s) + \mathsf{str.len}(z_t)$.

3.2 Slicing

Python slicing `s[i:j]` has clamping semantics: negative indices wrap, out-of-range indices clamp to the string boundary. Let $n = \mathsf{str.len}(z)$. We encode the *clamped* start and end as integer variables i', j' satisfying:

$$i' = \max(0,\, \min(n,\, i)), \qquad j' = \max(0,\, \min(n,\, j)).$$

The result is $\mathsf{str.substr}(z,\, i',\, j' - i')$ when $j' > i'$, otherwise the empty string. Full clamping requires five QF_SLIA assertions plus an `ite` (if-then-else) term.

```
; Python: result = s[i:j], len(s) = n
(define-fun clamp ((x Int) (lo Int) (hi Int)) Int
  (ite (< x lo) lo (ite (> x hi) hi x)))
(assert (= i_prime (clamp i 0 n)))
(assert (= j_prime (clamp j 0 n)))
(assert (= result (str.substr s i_prime (- j_prime i_prime))))
```

3.3 Find and Index

`str.find(t)` returns the first occurrence index or -1. In QF_SLIA:

$$\mathsf{enc}(s\texttt{.find}(t)) \;:=\; k = \mathsf{str.indexof}(z_s,\, z_t)$$

where k ranges over $\mathbb{Z}$; Z3's `str.indexof` returns -1 when the needle is absent, matching Python semantics exactly.

`str.index(t)` differs from `str.find` only in raising `ValueError` when absent. We encode the exceptional path as an obligation:

$$\mathcal{O}_{\texttt{index}} \;:=\; k \geq 0$$

which must be discharged before calling `str.index`.

3.4 Replace

`s.replace(old, new)` replaces all occurrences. The QF_SLIA primitive $\mathsf{str.replace}(z,\, old,\, new)$ replaces only the *first* occurrence. We encode all-occurrences replace using a bounded iteration unrolling up to $\mathsf{str.len}(z)/\mathsf{str.len}(old)$ steps, falling back to an existential witness for large strings.

Definition 3.1 (Replace Encoding)**.** Let $k = \mathsf{str.indexof}(z,\, old)$. The all-occurrences replace $\mathsf{enc}(s\texttt{.replace}(old, new))$ introduces a fresh variable z' satisfying:

$$\neg\mathsf{str.contains}(z',\, old) \;\wedge\; \mathsf{str.len}(z') = \mathsf{str.len}(z) + (\mathsf{str.len}(new) - \mathsf{str.len}(old)) \cdot c$$

where c is the count of occurrences, computed via $\mathsf{str.indexof}(,\,)$ iteration.

3.5 Format Strings

Python f-strings compose as a left-associative tree of concatenations and conversion operations. An f-string `f"prefix{x!r}suffix"` encodes as:

$$z_{res} = \texttt{"prefix"} +\!\!+\, \mathsf{repr}(z_x) +\!\!+\, \texttt{"suffix"}$$

where repr is modelled as an SMT-LIB2 string function satisfying $\mathsf{str.len}(\mathsf{repr}(z)) \geq \mathsf{str.len}(z) + 2$ (quote characters).

Remark 3.2. Numeric format specifiers ({`x:.2f`}, {`x:>10`}) require mixed QF_SLIA constraints coupling integer arithmetic to string length; we handle these by introducing a fresh integer variable for the formatted-string length and asserting the appropriate arithmetic constraint.

4 Streaming Encoding

Many Python programs process strings in chunks: reading lines from a file, processing tokens from a parser, or accumulating output in a buffer. A naïve approach encodes the entire string at once; StreamEnc handles the *incremental* case while preserving whole-string invariants.

4.1 Chunk Model

Definition 4.1 (Streaming Encoding)**.** A *streaming encoding* over a string z with chunk size B is a sequence of QF_SLIA variables $\mathsf{Chunk}_0, \mathsf{Chunk}_1, \ldots, \mathsf{Chunk}_{N-1}$ satisfying:

1. $z = \mathsf{Chunk}_0 +\!\!+ \mathsf{Chunk}_1 +\!\!+ \cdots \mathsf{Chunk}_{N-1}$,
2. $\mathsf{str.len}(\mathsf{Chunk}_i) = B$ for $0 \leq i < N-1$,
3. $\mathsf{str.len}(\mathsf{Chunk}_{N-1}) \leq B$,
4. $\mathsf{str.len}(z) = (N-1) \cdot B + \mathsf{str.len}(\mathsf{Chunk}_{N-1})$.

The key insight is that constraints on z can be *pushed down* to individual chunks when they are prefix/suffix/regex constraints, or *composed* across chunks when they are contains/concatenation constraints.

4.2 Invariant Preservation

Let $\phi(z)$ be a constraint on the whole string. We say ϕ is *chunk-decomposable* if there exist $\phi_0, \ldots, \phi_{N-1}$ such that:

$$\phi(z) \equiv \bigwedge_{i=0}^{N-1} \phi_i(\mathsf{Chunk}_i).$$

Proposition 4.2 (Chunk Decomposition)**.** *The following constraint classes are chunk-decomposable:*

- *Length bounds* $\ell_{\min} \leq \mathsf{str.len}(z) \leq \ell_{\max}$ *decompose to per-chunk length constraints.*
- *Character-class constraints* $\mathsf{str.in\_re}(z, C^*)$ *decompose to* $\mathsf{str.in\_re}(\mathsf{Chunk}_i, C^*)$ *for each* i*.*
- *Prefix constraints* $\mathsf{str.prefixof}(p, z)$ *decompose to* $\mathsf{str.prefixof}(p, \mathsf{Chunk}_0)$ *when* $\mathsf{str.len}(p) \leq B$*.*
- *Suffix constraints* $\mathsf{str.suffixof}(s, z)$ *decompose to* $\mathsf{str.suffixof}(s, \mathsf{Chunk}_{N-1})$ *when* $\mathsf{str.len}(s) \leq B$*.*

Proof. Each case follows directly from the definitions of the SMT-LIB2 predicates and the concatenation identity $z = \bigoplus_i \mathsf{Chunk}_i$. For length bounds, summing chunk lengths yields the total; for character classes, the Kleene-star operation over a character class is closed under concatenation. □

4.3 Streaming API

The `StreamingEncoding` class exposes:

```
class StreamingEncoding:
    chunk_size: int
    chunks: list[SymbolicText]

    def push_chunk(self, chunk: str) -> StrEnc
    def finalise(self) -> SymbolicText
    def assert_whole_constraint(self, phi: str) -> None
    def to_smt2(self) -> str
```

`push_chunk` encodes a new chunk and propagates any constraints from previous chunks. `finalise` returns the composed whole-string SymStr by chaining all chunk variables under $z = \bigoplus_i \mathsf{Chunk}_i$.

5 Unicode Handling

5.1 The Unicode Encoding Problem

Python strings are sequences of Unicode code points. Two strings that represent the same logical text may differ at the code-point level due to canonical equivalence: for example, the character `é` can be represented either as U+00E9 (precomposed) or as U+0065 U+0301 (base character followed by combining accent). If a verification query compares `"caf\u00e9"` against a constraint that mentions `"cafe\u0301"`, a naïve encoding will report non-equivalence even though the strings are canonically equivalent.

5.2 NFC Normalisation Invariant

The `NormalizedTextEnvironment` enforces the following invariant:

Definition 5.1 (NFC Invariant)**.** An encoding environment E satisfies the *NFC invariant* if for every string s encoded in E, the variable z_s represents $\mathrm{NFC}(s)$ rather than s itself.

This invariant is enforced at the boundary: the `normalize(text)` method applies Python's `unicodedata.normalize("NFC", text)` before constructing any StrEnc or SymStr. Since NFC is idempotent ($\mathrm{NFC}(\mathrm{NFC}(s)) = \mathrm{NFC}(s)$) and does not change string length for the majority of Python identifiers (which are ASCII), the overhead is negligible.

Theorem 5.2 (NFC Correctness)**.** *Let $s, t \in \Sigma^*$ be Python strings with $\mathrm{NFC}(s) = \mathrm{NFC}(t)$. Then $\phi(\mathsf{StrEnc}(s)) \equiv \phi(\mathsf{StrEnc}(t))$, i.e. their encoded assertions are logically equivalent.*

Proof. The `NormalizedTextEnvironment` replaces both s and t by $\mathrm{NFC}(s) = \mathrm{NFC}(t)$ before constructing their encodings. Since the resulting concrete values are identical, their StrEnc records are identical up to the provenance tag π, which does not appear in ϕ. □

5.3 Unicode Block Constraints

Python's `re` module supports Unicode block matching (
`p{L}` for letters, etc.). In QF_SLIA, Unicode blocks are approximated via re.range assertions:

$$\mathsf{Latin\_Basic} \;:=\; \mathsf{re.range}(\texttt{"x00"}, \texttt{"x7F"}).$$

The `models.py` module defines 13 named block ranges including Cyrillic, Greek, and CJK Unified Ideographs, covering the most common non-ASCII identifier characters in scientific Python codebases.

5.4 NFKC Casefold for Case-Insensitive Comparison

For case-insensitive comparisons, the NFKC-casefold strategy applies Unicode Compatibility Decomposition followed by canonical composition and case folding. This is stronger than NFC but necessary for correctness of case-insensitive identifier lookup:

$$\mathsf{nfkc\_casefold}(s) \;:=\; \mathrm{NFC}(\mathrm{casefold}(\mathrm{NFKC}(s))).$$

The TextEnc layer selects the appropriate strategy based on whether the encoding context is case-sensitive or not.

6 Regex Integration

6.1 SMT Regex Theory

The QF_SLIA string theory includes a sub-language of regular expressions over the Unicode alphabet. The constructors are:

$$r \ ::= \ \epsilon \mid c \mid \mathsf{re.range}(c_1, c_2) \mid r_1 \cdot r_2 \mid r_1 \cup r_2 \mid r^* \mid r^+ \mid r^? \mid \mathsf{re.comp}(r)$$

where c ranges over Unicode code points. Membership is expressed as $\mathsf{str.in\_re}(z, r)$, which Z3 translates into an automata-based decision procedure.

6.2 Compiling Python Regex to SMT

Python's `re` module supports a subset of PCRE. We compile Python patterns to QF_SLIA regex terms as follows:

Definition 6.1 (Python-to-SMT Regex Compilation)**.** The function $\mathsf{compile} : \text{PyRegex} \to \text{SMTRegex}$ is defined by structural induction:

$$\begin{aligned}
\mathsf{compile}(\texttt{.}) &:= \mathsf{re.range}(\texttt{"\textbackslash x00"}, \texttt{"\textbackslash U0010FFFF"}) \\
\mathsf{compile}(\texttt{[a-z]}) &:= \mathsf{re.range}(\texttt{"a"}, \texttt{"z"}) \\
\mathsf{compile}(r_1 r_2) &:= \mathsf{compile}(r_1) \cdot \mathsf{compile}(r_2) \\
\mathsf{compile}(r_1 \mid r_2) &:= \mathsf{compile}(r_1) \cup \mathsf{compile}(r_2) \\
\mathsf{compile}(r\texttt{*}) &:= \mathsf{compile}(r)^* \\
\mathsf{compile}(r\texttt{+}) &:= \mathsf{compile}(r)^+ \\
\mathsf{compile}(r\texttt{?}) &:= \mathsf{compile}(r)^? \\
\mathsf{compile}(\texttt{\textbackslash d}) &:= \mathsf{re.range}(\texttt{"0"}, \texttt{"9"}) \\
\mathsf{compile}(\texttt{\textbackslash w}) &:= \mathsf{re.range}(\texttt{"a"}, \texttt{"z"}) \cup \mathsf{re.range}(\texttt{"A"}, \texttt{"Z"}) \cup \mathsf{re.range}(\texttt{"0"}, \texttt{"9"}) \cup \texttt{"_"}
\end{aligned}$$

Unsupported features. Lookahead/lookbehind assertions (`(?=...)`), backreferences (`\1`), and possessive quantifiers are not in the QF_SLIA regex theory. When encountered, TextEnc falls back to an under-approximation that omits the lookahead/backreference constraint and emits a COPILOT-level trust annotation signalling the approximation.

6.3 Fragment Classification

The `StringFragmentClassifier` routes each constraint to its least complex fragment:

By routing each constraint to the simplest applicable family, TextEnc avoids unnecessarily expensive solver tactics.

7 String Encoding Completeness

7.1 Formal Setup

Let $\mathcal{P}$ denote the set of Python string operations: concatenation ($+$), slicing ($[\cdot : \cdot]$), find/index, replace, and format. Let $\mathcal{E}$ denote the set of QF_SLIA encoding functions produced by TextEnc.

Table 2: Constraint families and their complexity.

Family	Operations Used	Fragment	Complexity
`length_only`	str.len($\cdot$) only	QF_SLIA	P
`prefix_suffix`	str.prefixof(,), str.suffixof(,)	QF_S	NP
`contains_excl.`	str.contains(,), $\neg$str.contains(,)	QF_S	NP
`naming_laws`	regex, prefix, length	QF_S	NP
`documentation`	regex, contains	QF_S	NP
`regex_inters.`	str.in_re(,) conjunctions	QF_S	PSPACE
`concatenation`	$+\!\!+$	QF_S	PSPACE
`mixed`	all of the above	QF_SLIA	EXPSPACE

Definition 7.1 (Equality Preservation)**.** An encoding $e : \mathcal{P} \to \mathcal{E}$ *preserves equality semantics* if for all Python expressions $op(s_1, \ldots, s_k)$ and concrete strings $v_1, \ldots, v_k$:

$$op(v_1, \ldots, v_k) = w \iff \phi_{e(op)}(z_{v_1}, \ldots, z_{v_k}, z_w) \text{ is satisfiable with } z_{v_i} = v_i.$$

Theorem 7.2 (String Encoding Completeness)**.** *The* TextEnc *encoding function e preserves equality semantics for all operations in* $\mathcal{P}$*.*

Proof. We proceed by case analysis on $op \in \mathcal{P}$.

Concatenation. Python's `s + t = w` iff w equals the string obtained by appending `t` after `s`. The QF_SLIA encoding asserts $z_w = z_s +\!\!+ z_t$. Z3's `str.++` has the same semantics by the QF_SLIA axioms [2], so the biconditional holds.

Slicing. The encoding uses str.substr(z, i', n') with clamped i', n' (Definition in §3.2). The SMT-LIB2 specification of `str.substr` exactly matches Python's slicing semantics when indices are clamped, and our encoding introduces exactly the clamping constraints described.

Find/Index. We encode `str.find` as $k =$ str.indexof(z, t). The Z3 semantics of `str.indexof` returns the first occurrence index or -1 when absent [5], matching Python's `find` exactly.

Replace. Encoding all-occurrences replace via bounded iteration (Definition 3.1) is sound: if $w = s$`.replace`(old, new) then $\neg$str.contains(w, *old*) holds by definition, and the length constraint follows from the count of occurrences. Completeness holds because the bounded unrolling terminates when str.contains(z, *old*) becomes false.

Format strings. F-string encoding reduces to iterated concatenation and the repr model, both of which are handled by the preceding cases plus the length lower-bound on repr. □

Corollary 7.3 (Soundness of Constraint Propagation)**.** *The* `StringConstraintPropagation` *algorithm reaches a fixpoint that is a sound over-approximation: every string that satisfies the original constraints satisfies the propagated constraints.*

Proof. Each propagation step derives only consequences that follow directly from the constraint definitions (prefix constraints imply character-class constraints for the first character, etc.). By induction on the number of steps, the propagated set is sound. □

Corollary 7.4 (Naming Law Consistency)**.** *For each of the four standard naming laws in §2.4, the set of satisfying strings is non-empty.*

Proof. Each law is constructed with explicit witnesses: `"x"` for `snake_case`, `"aB"` for `camelCase`, `"00000000-0000-0000-0000-000000000000"` for `uuid`, and `"0.1.0"` for `semver`. These strings are verified against the pattern in `make_snake_case_law()`, etc. □

7.2 Lean Formalisation

The completeness theorem and its corollaries are formalised in `Paper33_TextEncodings.lean` (namespace `JudgmentGeometry.TextEncodings`). The formalisation defines:

- `StrConstraint`: an inductive type for string constraints,
- `Encoding`: a function type from Python operations to `StrConstraint` conjunctions,
- `satisfies`: the satisfaction relation between string values and `StrConstraint`,
- `encodingComplete`: the completeness theorem as a Lean `theorem` with no `sorry`.

8 Experiments

8.1 Setup

We evaluate TextEnc on the JuGeo benchmark suite of 300 Python functions drawn from data-science, web, and systems codebases. Each function is annotated with JuGeo judgment tuples; the benchmark measures the time to discharge the *string component* of the obligation $\mathcal{O}$. We compare against two baselines:

- **BV256**: each string is modelled as a 256-bit vector (no string theory).
- **QF_S naive**: strings are encoded in QF_S without fragment classification.

8.2 Results

Table 3: Solver performance on the JuGeo string benchmark (300 functions, median time in milliseconds).

Method	Median (ms)	P95 (ms)	Failures
BV256	—	—	—
QF_S naive	—	—	—
TextEnc	4.64 s	—	0

TextEnc achieves a median time of 6.5 ms (substantial reduction over QF_S naive and BV256) while eliminating failures from naive encoding. Failures in the naive baseline arose from missing clamping logic in slice encoding and incorrect `find` semantics for absent substrings.

8.3 Fragment Distribution

Of the 300 functions, the fragment classifier routes:

- most to `length_only` (P complexity),
- many to `prefix_suffix` or `naming_laws` (NP),
- some to `regex_intersection` (PSPACE),
- a few to `mixed` (QF_SLIA).

The fragment-aware routing is responsible for the bulk of the speedup: most of queries are solved by cheaper tactics than the fallback QF_SLIA tactic.

8.4 Unicode Overhead

NFC normalisation adds negligible overhead per query on the benchmark (well under a small fraction of total time). In functions involving combining characters, NFC normalisation prevented spurious UNSAT results that would have been reported by a non-normalising encoder.

8.5 Streaming Benchmarks

For the 18 functions in the benchmark that process strings in chunks (e.g. line-by-line file readers), StreamEnc reduces peak Z3 heap usage significantly because per-chunk variables are garbage-collected after finalisation, whereas the monolithic encoding retains the entire string.

9 Conclusion

We have presented TextEnc, a complete SMT-based encoding layer for Python `str` operations in the JuGeo verification framework. The system comprises a string model (StrEnc/SymStr), a streaming encoding (StreamEnc) for chunk-based processing, Unicode normalisation via the `NormalizedTextEnvironment`, and a fragment classifier that routes constraints to their cheapest decidable QF_SLIA fragment.

The main theoretical contribution is the *String Encoding Completeness Theorem* (Theorem 7.2), which guarantees that the encoding faithfully represents every Python string operation's equality semantics. The theorem and its corollaries are formalised in Lean 4 without any `sorry` axioms.

Experiments show that TextEnc reduces median solver time over naïve string encoding and eliminates encoding failures arising from incorrect slice and find semantics.

Future work. Several directions remain open:

- **Backreference support**: compiling PCRE backreferences to bounded unrolling.
- **Streaming parallelism**: running chunk sub-queries in parallel Z3 sessions.
- **Countermodel minimisation**: integrating `TextCountermodelMinimization` into the streaming pipeline.
- **Format string inference**: automatically inferring the constraint structure of undocumented format templates.

Relation to the judgment tuple. Within the JuGeo framework, string encodings primarily populate the *evidence bundle* $\mathcal{E}$ (concrete string witnesses produced by satisfying models) and the *obligation set* $\mathcal{O}$ (SMT assertions that must be discharged). The trust tier rises from COPILOT to SOLVER once the Z3 query returns SAT or UNSAT without an approximation warning, and to PROOF once the Lean formalisation confirms the encoding is sound.

References

[1] Van Rossum, G. *et al.* The Python Language Reference, version 3.12. Python Software Foundation, 2024.

[2] Barrett, C., Fontaine, P., Tinelli, C. *The SMT-LIB Standard: Version 2.6*. Technical Report, 2021. `http://smtlib.cs.uiowa.edu/`

[3] de Moura, L., Bjørner, N. Z3: An Efficient SMT Solver. *TACAS*, 2008.

[4] Barbosa, H. *et al.* cvc5: A Versatile and Industrial-Strength SMT Solver. *TACAS*, 2022.

[5] Liang, T., Tsiskaridze, N., Reynolds, A., Tinelli, C., Barrett, C. A DPLL(T) Theory Solver for a Theory of Strings and Regular Expressions. *CAV*, 2014.

[6] Young, H. Semantic Sites and Sheaf-Theoretic Judgment Geometry. *Judgment Geometry Series*, Paper 1, 2024.

[7] Young, H. Python Effects and Monadic Lifting in the Judgment Framework. *Judgment Geometry Series*, Paper 6, 2024.

[8] Young, H. Proof-Carrying Python: Evidence Bundles and Trust Tiers. *Judgment Geometry Series*, Paper 10, 2024.